

~/Desktop/Unito/Algoritmi e StruttureDati/immagini pdf/Codici_uniti.c

```
1 void Bandiera(Color B[], int n) {
2     int i = 0, j = 0;
3     for (int k = 0; k < n; k++) {
4         if (B[k] == bianco) {
5             swap(B, k, j);
6             j++;
7         }
8         if (B[k] == verde) {
9             swap(B, k, j);
10            swap(B, j, i);
11            j++;
12            i++;
13        }
14    }
15 }
16
17 int countBubbleSort(int A[], int n) {
18     int count = 0;
19     bool sw = true;
20     for (int i = n - 1; i > 0 && sw; i--) {
21         sw = false;
22         for (int j = 0; j < i; j++) {
23             count += 1;
24             if (A[j] > A[j + 1]) {
25                 swap(A, j, j + 1);
26                 sw = true;
27             }
28         }
29     }
30     return count;
31 }
32
33 int find(int A[], int N, int f) {
34     int m = 0;
35     int n = N - 1;
36     while (m < n) {
37         int i = m;
38         int j = n;
39         int r = A[f];
40         while (i <= j) {
41             while (A[i] < r) i++;
42             while (A[j] > r) j--;
43             if (i <= j) {
44                 swap(A, i, j);
45                 i++;
46                 j--;
47             }
48         }
49     }
```

```
49     if (f <= j) n = j;
50     else if (f >= i) m = i;
51     else break;
52 }
53 return A[f];
54 }
55
56 int partition(int A[], int P, int r) {
57     int i = P + 1;
58     int j = r;
59     int x = A[P];
60     while (i <= j) {
61         if (A[i] <= x) i++;
62         else if (A[j] > x) j--;
63         else {
64             swap(A, i, j);
65             i++;
66             j--;
67         }
68     }
69     int q = i - 1;
70     swap(A, P, q);
71     return q;
72 }
73
74 void quickSort(int A[], int i, int j) {
75     if (i < j) {
76         int p = partition(A, i, j);
77         quickSort(A, i, p - 1);
78         quickSort(A, p + 1, j);
79     }
80 }
81
82 int maxLen(int v[], int size) {
83     int count = 1;
84     int max = 1;
85     for (int i = 1; i < size; i++) {
86         if (v[i - 1] > v[i]) count++;
87         else if (count > max){
88             max = count;
89             count = 1;
90         }
91     }
92     if (count > max) {
93         max = count;
94         count = 1;
95     }
96     return max;
97 }
98
```

```
99  int searchRec(int a[], int i, int target) {
100      int j = size(a);
101      if (i > j) return -1;
102      if (a[i] == target) return i;
103      else return searchRec(a, i + 1, target);
104  }
105
106  int searchIter(int a[], int i, int target) {
107      while (i < size(a) && a[i] != target) i++;
108      if (i >= size(a)) return -1;
109      else return i;
110  }
111  int dicotomicSearchRec(int A[], int i, int j, int target){
112      if(i > j) return -1;
113      int m = (i+j)/2;
114      if(A[m] == target) return m;
115      else if(A[m] < target) return dicotomicSearchRec(A, m+1, j, target);
116      else return dicotomicSearchRec(A, i, m-1, target);
117  }
118
119  int dicotomicSearchIter(int A[], int i, int j, int target){
120      int l = i;
121      int r = j;
122      while (l <= r) {
123          int m = (l+r)/2;
124          if(A[m] == target) return m;
125          else if(A[m] < target) l = m+1;
126          else r = m-1;
127      }
128      return -1;
129  }
130  int expRic(int x, int n){
131      if(n == 0){
132          return 1;
133      }
134      else {
135          int y = expRec(x, n/2);
136          if(n % 2 == 0) {
137              return y * y;
138          }
139          else {
140              return y * y * x;
141          }
142      }
143  }
144
145  int expIter(int x, int n){
146      int y = x;
147      int k = n;
148      int z = 1;
```

```
149     while(k > 0){
150         if(k % 2 == 1) {
151             z = z * y;
152         }
153         y = y * y;
154         k = k / 2;
155     }
156     return z;
157 }
158 void minHeapInsert(int H[], int x) {
159     H.size = H.size+1;
160     int hd = H.size - 1;
161     int i = hd; // ultimo elemento dell'array
162     H[hd] = x;
163     while (i > 0 && H[parent(i,hd)] > H[i]) {
164         swap(H, parent(i, hd), i);
165         i = parent(i, hd);
166     }
167 }
168
169 void maxHeapInsert(int H[], int x) {
170     H.size = H.size+1;
171     int hd = H.size - 1;
172     int i = hd; // ultimo elemento dell'array
173     H[hd] = x;
174     while (i > 0 && H[parent(i,hd)] < H[i]) {
175         swap(H, parent(i, hd), i);
176         i = parent(i, hd);
177     }
178 }
179
180 int extractMin(int H[]) {
181     int res = H[0];
182     H[0] = H[H.size - 1];
183     H.size -= 1;
184     minHeapify(H, 0, H.size-1);
185 }
186
187 int extractMax(int H[]) {
188     int res = H[0];
189     H[0] = H[H.size - 1];
190     H.size -= 1;
191     maxHeapify(H, 0, H.size-1);
192 }
193 void maxHeapify(int H[], int i, int hd){
194     int max = (H[left(i, hd)] >= H[right(i, hd)]) ? left(i, hd) : right(i, hd);
195     max = (H[max] > H[i]) ? max : i;
196     if(max != i){
197         swap(H, i, max);
198         maxHeapify(H, max, hd);
199     }
```

```
199     }
200 }
201
202 void minHeapify(int H[], int i, int hd){
203     int min = (H[left(i, hd)] <= H[right(i, hd)]) ? left(i, hd) : right(i, hd);
204     min = (H[min] < H[i]) ? min : i;
205     if(min != i){
206         swap(H, i, min);
207         minHeapify(H, min, hd);
208     }
209 }
210 void buildMinHeap(int A[], int dim) {
211     int m = dim / 2 + 1;
212     for(int i = m; i >= 0; i--)
213         minHeapify(A, i, dim);
214 }
215
216 // Pre: hd < dim A[]
217 // Post: A[0..hd] è uno heap massimo
218 void buildMaxHeap(int A[], int dim) {
219     int m = dim/2 + 1;
220     for(int i = m; i >= 0; i--)
221         maxHeapify(A, i, dim);
222 }
223 void heapSort(int A[], int dim) {
224     int hd = dim - 1;
225     buildMaxHeap(A, hd);
226     for (int i = hd; i > 0; i--) {
227         swap(A, 0, i);
228         maxHeapify(A, 0, i-1);
229     }
230 }
231 list mergeNonDistruttiva(list l, list m) {
232     if(l == NULL)
233         return m;
234     else if(m == NULL)
235         return l;
236     else if(l->info <= m->info) {
237         return Cons(l->info, merge(l->next, m));
238     } else { // l->info > m->info
239         return Cons(m->info, merge(l, m->next));
240     }
241 }
242 list merge(list l, list m) {
243     if(l == NULL)
244         return m;
245     else if(m == NULL)
246         return l;
247     else if(l->info <= m->info) {
248         l->next = merge(l->next, m);
```

```
249     return l;
250 } else { // l->info > m->info
251     m->next = merge(l, m->next);
252     return m;
253 }
254 }
255 list split(list l) {
256     list slow = l;
257     list fast = l->next;
258     while (fast && fast->next) {
259         slow = slow->next;
260         fast = fast->next->next;
261     }
262     list secondHalf = slow->next;
263     slow->next = NULL;
264     return secondHalf;
265 }
266 list mergeSort(list l) {
267     if (l == NULL || l->next == NULL)
268         return l;
269     else {
270         list m = split(l); // divide la lista in due
271         l = mergeSort(l); // ordina una metà
272         m = mergeSort(m); // ordina l'altra metà
273         return merge(l, m);
274     }
275 }
276 list unione(list l, list m) {
277     if(l == NULL) return m;
278     if(m == NULL) return l;
279     else if(l->info == m->info)
280         return Cons(l->info, unione(l->next, m->next));
281     else if(l->info < m->info)
282         return Cons(l->info, unione(l->next, m));
283     else return Cons(m->info, unione(l, m->next));
284 }
285
286 list intersezione(list l, list m) {
287     if(l == NULL) return m;
288     else if(l->info == m->info)
289         return Cons(l->info, intersezione(l->next, m->next));
290     else if(l->info < m->info)
291         return intersezione(l->next, m);
292     else return intersezione(l, m->next);
293 }
294
295 list differenza(list l, list m) {
296     if(l == NULL) return m;
297     if(m == NULL) return l;
298     else if(l->info == m->info)
```

```
299     return differenza(l->next, m->next);
300 else if(l->info < m->info)
301     return Cons(l->info, differenza(l->next, m));
302 else return differenza(l, m->next);
303 }
304
305 list symmDiff(list l, list m) {
306     if(l == NULL) return m;
307     if(m==NULL) return l;
308     else if(l->info == m->info)
309         return symmDiff(l->next, m->next);
310     else if(l->info < m->info)
311         return Cons(l->info, symmDiff(l->next, m));
312     else return Cons(m->info, symmDiff(l, m->next));
313 }
314
315 int rank(list l) {
316     if(l==NULL) return 0;
317     else{
318         l->info += rank(l->next);
319         return l->info;
320     }
321 }
322
323 int corank_aux(list l, int cumulativeCoRank) {
324     if(l==NULL) return cumulativeCoRank;
325     else{
326         int tmp = l->info;
327         l->info += cumulativeCoRank;
328         return corank_aux(l->next, cumulativeCoRank + tmp);
329     }
330 }
331 int corank(list l){return corank_aux(l, 0);}
332
333 list reverse(list l) {
334     list reversed = NULL;
335     while(l != NULL){
336         reversed = Cons(l->info, reversed);
337         l=l->next;
338     }
339     return reversed;
340 }
341
342 list fast_reverse_aux(list l, list m){
343     if (l==NULL) return m;
344     else return fast_reverse_aux(l->next, Cons(l->info, m));
345 }
346 list fast_reverse(list l){return fast_reverse_aux(l, NULL);}
347
348 bool equal(list l, list r){
```

```
349     if(l==NULL) return r==NULL;
350     else if(r==NULL) return l==NULL;
351     else return l->info == r->info && equal(l->next, r->next);
352 }
353
354 bool palindrome(list l){
355     if(l==NULL) return true;
356     list reversed = reverse(l);
357     return equal(l, reversed);
358 }
359
360 list deleteAll(int n, list as){
361     if(as == NULL) return NULL;
362     else if(as->info == n){
363         list ts = as->next;
364         free(as);
365         return deleteAll(n, ts);
366     }else{
367         as->next= deleteAll(n, as->next);
368         return as;
369     }
370 }
371
372 list insert(list as, list bs, int n){
373     if(n==0) return concat(as,b);
374     else{
375         bs->next = insert(as, bs->next, n-1);
376         return bs;
377     }
378 }
379
380 list odd_aux(list l){
381     if(l==NULL) return l;
382     else if(l->info % 2 == 1)
383         return Cons(l->info, odd_aux(l->next));
384     else return odd_aux(l->next);
385 }
386
387 list even_aux(list l){
388     if(l==NULL) return l;
389     else if(l->info % 2 == 0)
390         return Cons(l->info, even_aux(l->next));
391     else return even_aux(l->next);
392 }
393
394 list oddFirst(list l){return concat(odd_aux(l), even_aux(l));}
395
396 Triple isOrderedBTreeAux(btrees bt){
397     if(bt->left == NULL && bt->right == NULL){ // isLeaf(bt)
398         t.isOrdered = true;
```



```
399     t.min = t.max = bt->key;
400 }else if(bt->right == NULL){ // bt->left != NULL
401     Triple tleft = isOrderedBTreeAux(bt->left);
402     t.isOrdered = tleft.isOrdered && bt->key > tleft.max;
403     t.min = tleft.min;
404     t.max = bt->key;
405 }else if(bt->left == NULL){ // bt->right != NULL
406     Triple tright = isOrderedBTreeAux(bt->right);
407     t.isOrdered = tright.isOrdered && bt->key < tright.min;
408     t.min = bt->key;
409     t.max = tright.max;
410 }else{ // bt->left != NULL && bt->right != NULL
411     Triple tleft = isOrderedBTreeAux(bt->left);
412     Triple tright = isOrderedBTreeAux(bt->right);
413     t.isOrdered = tleft.isOrdered && tright.isOrdered && bt->key > tleft.max && bt-
>key < tright.min;
414     t.min = tleft.min;
415     t.max = tright.max;
416 }
417 return t;
418 }
419
420 bool isOrdered(btree bt){
421     if(bt == NULL) return true;
422     else{
423         Triple t = isOrderedBTreeAux(bt);
424         return t.isOrdered;
425     }
426 }
427
428 list DescList_aux(btree bt, list l){
429     if(bt == NULL) return l;
430     else{
431         l = DescList_aux(bt->left, l);
432         l = Cons(bt->key, l);
433         return DescList_aux(bt->right, l);
434     }
435 }
436
437 list DescList(btree bt){return DescList_aux(bt, NULL);}
438
439 list Cresclist_aux(btree bt, list l) {
440     if(bt == NULL) return l;
441     else {
442         l = Cresclist_aux(bt->right, l);
443         l = Cons(bt->key, l);
444         return Cresclist_aux(bt->left, l);
445     }
446 }
447
```

```
448 list Cresclist(btree bt) {
449     return Cresclist_aux(bt, NULL);
450 }
451
452 btree rightAncestor(btree nd) {
453     btree p = nd->parent;
454     while (p != NULL && nd == p->right) {
455         nd = p;
456         p = nd->parent;
457     }
458     return p;
459 }
460
461 btree successor(btree nd) {
462     if(nd->right != NULL) return minInBitree(nd->right);
463     else return rightAncestor(nd);
464 }
465
466 btree maxInBitree(btree bt) {
467     while (bt->right != NULL) {
468         bt = bt->right;
469     }
470     return bt;
471 }
472
473 btree minInBitree(btree bt) {
474     while (bt->left != NULL) {
475         bt = bt->left;
476     }
477     return bt;
478 }
479
480 int cardinality(btree t) {
481     if(t == NULL) return 0;
482     int l, r;
483     l = cardinality(t->left);
484     r = cardinality(t->right);
485     return l + r + 1;
486 }
487
488 int height(btree node) {
489     if(node->left == NULL && node->right == NULL) return 0;
490     else {
491         int hl = 0, hr = 0;
492         if(node->left != NULL) hl = height(node->left);
493         if(node->right != NULL) hr = height(node->right);
494         return max(hl, hr) + 1;
495     }
496 }
497
498 void bTreeCrescente(btree bt) {
```

```
498     if(bt == NULL) return;
499     bTreeCrescente(bt->left);
500     printf("%d ", bt->key);
501     bTreeCrescente(bt->right);
502 }
503
504 void bTreeDecrescente(btree bt) {
505     if(bt == NULL) return;
506     bTreeDecrescente(bt->right);
507     printf("%d ", bt->key);
508     bTreeDecrescente(bt->left);
509 }
510
511 btree insert(int k, btree bt) {
512     if(bt == NULL) return ConsTree(k, NULL, NULL);
513     else if(k == bt->key) return bt;
514     else if(k > bt->key) {
515         bt->right = insert(k, bt->right);
516         return bt;
517     } else { // k < bt->key
518         bt->left = insert(k, bt->left);
519         return bt;
520     }
521 }
522
523 btree antenatoComune(btree bt, int a, int b) {
524     if(bt->key >= a && bt->key <= b) return bt;
525     else if(bt->key < a) return antenatoComune(bt->right, a, b);
526     else if(bt->key > b) return antenatoComune(bt->left, a, b);
527 }
528
529 int height(kTree t) {
530     if(t == NULL) return 0;
531     else if(t->child == NULL) return 0;
532     else{
533         int h = 0;
534         kTree c = t->child;
535         while(c != NULL){
536             h = max(h, height(c));
537             c = c->sibling;
538         }
539         return h+1;
540     }
541 }
542
543 int sumLeaf(kTree t) {
544     if(t == NULL) return 0;
545     else if(t->child == NULL) return t->key;
546     else{
547         int sum = 0;
```

```
548     kTree c = t->child;
549     while (c != NULL){
550         sum += sumLeaf(c);
551         c = c->sibling;
552     }
553     return sum;
554 }
555
556 }
557
558 list fringe(kTree t) {
559     list l = NULL;
560     if(t == NULL) return NULL;
561     else if(t->child == NULL) // t è una foglia
562         return Cons(t->key, NULL);
563     else{ // t NON è una foglia
564         kTree c = t->child;
565         while (c != NULL){
566             l = concat(l, fringe(c));
567             c = c->sibling;
568         }
569     }
570     return l;
571 }
572
573 int cardinality(kTree t) {
574     if(t==NULL) return 0;
575     else{
576         int card = 1;
577         kTree c = t->child;
578         while (c != NULL){
579             card += cardinality(c);
580             c = c->sibling;
581         }
582         return card;
583     }
584 }
585
586 int degreeProf(kTree t) {
587     if(t==NULL) return 0;
588     else if(t->child == NULL) return 0;
589     else{
590         kTree c = t->child;
591         int droot = 0;
592         int dt = 0;
593         while(c != NULL){
594             dt = max(dt, degree(c));
595             droot = droot + 1;
596             c = c->sibling;
597         }
```

```
598     return max(dt, droot);
599 }
600 }
601
602 int degree(kTree t) {
603     if(t==NULL) return 0;
604     else{
605         int num_degree = 0;
606         kTree c = t->child;
607         while (c!=NULL){
608             num_degree = max(num_degree, degree(c));
609             c=c->sibling;
610         }
611         return num_degree+1;
612     }
613 }
614
615 list KTreeBFS(kTree t) {
616     if(t == NULL) return NULL;
617     list l = NULL;
618     queue q = NewQueue();
619     EnQueue(t, q);
620     while (!isEmptyQueue(q)) {
621         kTree node = DeQueue(q);
622         l = Cons(node->key, l);
623         node = node->child;
624         while(node != NULL) {
625             EnQueue(node, q);
626             node = node->sibling;
627         }
628     }
629     return reverse(l);
630 }
631
632 void sommaCammino(kTree t, int s){
633     if(t->child == NULL){
634         t->child = consTree(s + t->key, NULL, NULL);
635     }else{
636         s+=t->key;
637         kTree c = t->child;
638         while (c != NULL) {
639             sommaCammino(c, s);
640             c=c->sibling;
641         }
642     }
643 }
644
645 void sommaRamo(kTree t){
646     sommaCammino(t,0);
647 }
```

```
648
649 int maxSumBranch(kTree t) {
650     if(t == NULL) return 0;
651     else if(t->child == NULL) return t->key;
652     else{
653         int maxSum = 0;
654         kTree c = t->child;
655         while (c != NULL) {
656             maxSum = max(maxSum, maxSumBranch(c));
657             c=c->sibling;
658         }
659         return t->key + maxSum;
660     }
661 }
662
663 kTree completeBedi(int key, int dg, int ht) {
664     if(ht==0) return constTree(key, NULL, NULL);
665     kTree child= complete(key, dg, ht-1);
666     kTree temp = child;
667     for(int i = 1; i<dg; i++) {
668         temp->sibling = complete(key, dg, ht-1);
669         temp = temp->sibling;
670     }
671     return constTree(key, child, NULL);
672 }
673
674 bool sum(kTree t) {
675     if(t->child == NULL) return true;
676     kTree c = t->child;
677     int res = 0;
678     bool b = true;
679     while (c != NULL){
680         res += c->key;
681         b = b & sum(c);
682         c=c->sibling;
683     }
684     return t->key == res && b;
685 }
686
687 int nodiProfondi(kTree t, int h) {
688     if(h == 0) return 1;
689     else{
690         int n = 1;
691         kTree c = t->child;
692         while (c!=NULL){
693             n += nodiProfondi(c, h-1);
694             c = c->sibling;
695         }
696         return n;
697     }
```

```
698 }
699
700 int shortest2(kTree t) {
701     if(t==NULL) return 0;
702     else if(t->child == NULL) return 1;
703     else{
704         int corto = 10000;
705         kTree c = t->child;
706         while (c!=NULL){
707             int pathlength = corto = min(corto, shortest(c));
708             c=c->sibling;
709         }
710         return corto+1;
711     }
712 }
713
714 int countInternalNodes(kTree t){
715     if(t == NULL) return 0;
716     else if(t->child == NULL) return 0;
717     int sumInternalNodes = 0;
718     kTree c = t->child;
719     while(c != NULL){
720         sumInternalNodes += countInternalNodes(c);
721         c = c->sibling;
722     }
723     return sumInternalNodes+1;
724 }
725
726 bool isOdd(kiTree t) {
727     if (t == NULL)
728         return false;
729     else if (t->child == NULL) {
730         if (t->child->key % 2 == 1) // a
731             return true;
732         else // t->child->key % 2 == 0
733             return false;
734     }
735     else {
736         bool odd = true;
737         kTree c = t->child;
738         while (c != NULL) {
739             odd = odd && isOdd(c);
740             c = c->sibling;
741         }
742         return odd;
743     }
744 }
745
746 int minimo(kiTree t) {
747     if (t == NULL)
```

```
748     return NULL;
749     if (t->child == NULL)
750         return t->key;
751     int fm = t->key;
752     kTree currentChild = t->child;
753     while (currentChild != NULL) {
754         fm = min(fm, minimo(currentChild));
755         currentChild = currentChild->sibling;
756     }
757     return fm;
758 }
759
760 int Large(kiTree t){
761     if (t == NULL) return 0;
762     queue q = NewQueue();
763     EnQueue(t, q);
764     int max = 0;
765     int count = 0;
766     while (!isEmptyQueue(q)){
767
768         kTree current = DeQueue(q);
769         kTree c = current->child;
770         count = 0;
771         while (c != NULL){
772             EnQueue(c, q);
773             count += 1;
774             c = c->sibling;
775         }
776         if (count > max){
777             max = count;
778             count = 0;
779         }
780     }
781     return max + 1;
782 }
783 int hashInsert(HashTable T, int k) {
784     for (int i = 0; i < T->dim; i++) {
785         int pos = LinearProbing(k, i, T->dim);
786         if (pos >= T->dim) return -2;
787         // se è già presente ritorna -1
788         if (T->array[pos] == k) return -1;
789         // inserimento
790         if (T->array[pos] == -1) {
791             T->array[pos] = k;
792             return pos;
793         }
794     }
795     return -2;
796 }
797 int hashSearch(HashTable T, int k) {
```



```
798     for (int i = 0; i < T->dim; i++) {
799         int pos = LinearProbing(k, i, T->dim);
800         if (pos >= T->dim) return -2;
801         // se è presente ritorna l'elemento
802         if (T->array[pos] == k) return T->array[pos];
803         // se non è presente ritorna -1
804         if (T->array[pos] == -1) return -1;
805     }
806     return -2;
807 }
808
809 list ordInsert(int k, list p, list l) {
810     if (l == NULL) { // La Lista è vuota
811         return Cons(k, p, NULL);
812     } else if (k == l->info) {
813         // Element already exists, don't insert duplicate
814         return l;
815     } else if (k < l->info) {
816         return Cons(k, p, l);
817     } else {
818         l->next = ordInsert(k, l, l->next);
819         return l;
820     }
821 }
822
823 void hashInsert(HashTable T, int k) {
824     int pos = hashFun(k, T->dim);
825     T->array[pos] = ordInsert(k, NULL, T->array[pos]);
826 }
827
828 int hashSearch(HashTable T, int k) {
829     int pos = hashFun(k, T->dim);
830     while(T->array[pos] != NULL){
831         if (T->array[pos]->info == k) return T->array[pos]->info;
832         else T->array[pos] = T->array[pos]->next;
833     }
834     return -1;
835 }
836
837 void decreaseKey(Heap H, int i, int newKey) {
838     H->keys[i] = newKey;
839     int par = parent(H, i);
840     if (par != i && H->keys[i] < H->keys[par]) {
841         swap(H, i, par);
842         decreaseKey(H, par, H->keys[par]);
843     }
844 }
845
846 bool subset(list l, list m) {
847     if (l == NULL)
```

```
848     return true;
849     if (m == NULL)
850         return false;
851     if (l->info == m->info)
852         return subset(l->next, m->next);
853     if (l->info > m->info)
854         return subset(l, m->next);
855     return false;
856 }
857
858 void OddEven(int A[], int n) {
859     int left = 0, right = n - 1;
860     while (left < right) {
861         while(A[left] % 2 != 0){
862             left++;
863         }
864         while (A[right] % 2 == 0){
865             right--;
866         }
867         if (left < right) {
868             swap(A, left, right);
869             left++;
870             right--;
871         }
872     }
873 }
874
875 int degree(kTree t) {
876     // Precondizione: t non è vuoto
877     // Postcondizione: ritorna il grado di t, ossia
878     // il massimo numero di figli di un vertice in t
879     if (t == NULL) return 0;
880     int num_degree = 0;
881     kTree child = t->child;
882     while (child != NULL) {
883         int childDegree = degree(child);
884         num_degree = (childDegree > num_degree) ? childDegree : num_degree;
885         child = child->sibling;
886     }
887     return num_degree + 1;
888 }
889
890 list diff(list l1, list l2) {
891     if (l1 == NULL) return NULL; // Se l1 è vuoto, il risultato è NULL
892
893     // Avanziamo in l2 finché i suoi elementi sono minori di l1->info
894     while (l2 != NULL && l2->info < l1->info) {
895         l2 = l2->next;
896     }
897 }
```

```
898 // Se i due elementi sono uguali, saltiamo l1 e l2
899 if (l2 != NULL && l1->info == l2->info) {
900     return diff(l1->next, l2->next);
901 }
902
903 // Se l1->info non è in l2, lo aggiungiamo al risultato
904 list result = new list1;
905 result->info = l1->info;
906 result->next = diff(l1->next, l2);
907 return result;
908 }
909
910 void merge(int A[], int a, int p, int b) {
911     int n1 = p - a + 1; // Dimensione del primo sottoarray
912     int n2 = b - p;      // Dimensione del secondo sottoarray
913
914     int *L = (int *)malloc(n1 * sizeof(int));
915     int *R = (int *)malloc(n2 * sizeof(int));
916
917     arraycpy(A, a, p, L, 0); // Copia A[a..p] in L[0..n1-1]
918     arraycpy(A, p + 1, b, R, 0); // Copia A[p+1..b] in R[0..n2-1]
919
920     int i = 0, j = 0, k = a;
921
922     while (i < n1 && j < n2) {
923         if (L[i] <= R[j]) {
924             A[k++] = L[i++];
925         } else {
926             A[k++] = R[j++];
927         }
928     }
929
930     while (i < n1) {
931         A[k++] = L[i++];
932     }
933
934     while (j < n2) {
935         A[k++] = R[j++];
936     }
937
938     free(L);
939     free(R);
940 }
```